



**UNIVERSITY OF
CAMBRIDGE**

**Development of a Machine Learning Model
to Estimate Downtime from Cybersecurity
Breaches in Industrial Networks**

Name of Student:

Department:

Registration No.:

Name of Research Supervisor:

Abstract

This dissertation conducts research on how predictive machine learning can be designed to predict downtime in industrial networks after cybersecurity attacks. The nature of Industry Control Systems (ICS) and Operational Technology (OT) infrastructures increasingly falls prey to advanced cyberattacks, and the current body of research focuses on detection but there is a lack of focus on post-attack downtime prediction. This research fills that gap by using supervised machine learning models; Random Forest, XGBoost and Long Short-Term Memory (LSTM) to predict the time of downtime using the ICSSIM dataset available at Kaggle; it includes simulated events of the breach, system parameters and recovery time. Preprocessing of the dataset to deal with the problems of missing values, outliers, and categorical encodings was performed, as was the preliminary exploratory data analysis and feature engineering, including temporal and system-load-related features. Training and validation of models was done in Python with MATLAB being incorporated to create a user-friendly graphical interface to estimate real-time downtime. Predictive ability was measured by the determination of evaluation measures like Root Mean Squared Error (RMSE), Mean Absolute Error (MAE) and R^2 . Comparative analysis showed that ensemble-based models were more robust and generalized as compared to the other models, whereas the LSTM was able to incorporate temporal dependencies. Analytical power of Python and visualization power of MATLAB led to the creation of a hybrid tool to be used in industry. Results indicate that down-time prediction models would be useful in planning resiliency, minimizing recovery duration and offering actionable intelligence to incident response teams. This research paper will be useful in improving the concept of cybersecurity resilience in the industrial environment; it will allow closing the gap between the detection-oriented tools and operational continuity management.

Acknowledgement

I would like to highly appreciate my supervisor who provided me with invaluable advice, my department that provided me with resources and assistance, and my family that supported me throughout this research undertaking and made this research work successful.

List of Acronyms

Acronym	Full Form
ICS	Industrial Control System
OT	Operational Technology
SCADA	Supervisory Control and Data Acquisition
ML	Machine Learning
RF	Random Forest
XGBoost	Extreme Gradient Boosting
LSTM	Long Short-Term Memory
GUI	Graphical User Interface
RMSE	Root Mean Squared Error
MAE	Mean Absolute Error
R ²	Coefficient of Determination
API	Application Programming Interface
DoS	Denial of Service
APT	Advanced Persistent Threat

Table of Contents

Abstract	2
Acknowledgement	3
List of Acronyms	4
Chapter 1: Introduction	13
1.1 Aim	13
1.2 Objectives	13
1.3 Problem Statement	13
1.4 Research Questions	14
1.5 Research Area	14
1.6 Expected Practical Element Output.....	16
1.7 Required Resources	17
1.8 Prerequisite Knowledge/Skills Required	18
1.9 Project Plan – Gantt Chart	18
Chapter 2: Literature Review/Related Work	19
2.1 Comprehensive Overview of the Existing Literature	19
2.1.1 Evolution of Cybersecurity Threats in Industrial Systems	19
2.1.2 Machine Learning Applications in Cyber Threat Analysis	20
2.1.3 Predictive Modeling for Downtime Estimation	22
2.1.4 Integration of Python and MATLAB for Hybrid Implementation	23
2.1.5 Use of Public Datasets for Industrial Cybersecurity Research	24
2.1.6 Visualization and Interpretability in Downtime Prediction Systems	25
2.2 Critical Analysis of Existing Studies	27
2.2.1 Focus on Detection Rather Than Prediction	27
2.2.2 Lack of Downtime-Specific Datasets	28

2.2.3 Integration Challenges in Hybrid Environments	29
2.2.4 Generalizability Across Industrial Systems	30
2.2.5 Limited Attention to User-Centric Design.....	31
2.2.6 Literature Gap.....	32
2.3 Policy, Compliance and Ethics	33
2.4 Dataset Limitations.....	33
2.5 Performance Metrics in Comparison Tables.....	34
2.6 Integration Challenges – Organizational Adoption.....	34
2.7 Summary	35
Chapter 3: Methodology.....	36
3.1 Research Design.....	36
3.2 Data Collection and Preprocessing.....	37
3.2.1 Data Sources and Provenance	38
3.2.2 Data Structure and Variables.....	38
3.2.3 Target Variable: Downtime Duration	39
3.2.4 Data Quality and Limitations	39
3.3 Exploratory Data Analysis.....	40
3.3.1 Univariate and Bivariate Analysis	40
3.3.2 Correlation and Multicollinearity Assessment.....	40
3.4 Feature Engineering.....	41
3.4.1 Temporal Feature Extraction.....	41
3.4.2 Feature Scaling	41
3.5 Model Development	42
3.5.1 Random Forest.....	42
3.5.2 XGBoost.....	42

3.5.3 Long Short-Term Memory (LSTM)	42
3.6 Model Evaluation Metrics	43
3.7 Ethical Considerations	43
3.8 Conclusion	43
Chapter 4: Implementation and Development	45
4.1 Environment Setup and Library Installation	45
4.1.1 Core Library Imports	45
4.2 Kaggle Authentication Setup	46
4.3 Dataset Acquisition and Extraction	47
4.3.1 Dataset File Management	48
4.4 Data Loading and Label Detection	48
4.4.1 Exploratory Data Analysis Preparation	49
4.5 Global Parameter Configuration	50
4.6 Preprocessing for Classification	51
4.6.1 Classification Models	52
4.6.2 Cross-Validation for Classification	52
4.6.3 Final Model Training and Saving	53
4.6.4 Classification Plot Generation	53
4.7 Preprocessing for Regression	54
4.7.1 Regression Models	55
4.7.2 Cross-Validation for Regression	56
4.7.3 Final Regression Training and Saving	57
4.7.4 Regression Plot Generation	57
4.8 Feature Importance Extraction	59
4.9 Model Saving and Management	61

4.10 Chapter Conclusion	61
Chapter 5: Results and Evaluation	63
5.1 Classification Model Results	63
5.1.1 Accuracy and Metrics	64
5.1.2 Confusion Matrix and ROC Analysis	65
5.2 Regression Model Results	66
5.2.1 Performance Metrics (RMSE, MAE, R^2).....	67
5.2.2 Error Distribution and Residual Plots	68
5.3 Comparative Analysis of Models	70
5.4 Comparative Analysis.....	71
5.4.1 Comparison with Existing Models	71
5.4.2 Strengths of the Proposed Approach	71
5.4.3 Limitations Identified	72
5.5 Critical Evaluation.....	72
5.5.1 Technical Evaluation	72
5.5.2 Practical Evaluation	72
5.5.3 Ethical and Future Considerations.....	73
5.6 Chapter Conclusion	73
Chapter 6: Conclusion and Recommendation	75
6.1 Introduction.....	75
6.2 Justification of Aim and Objectives	75
6.3 Summary of Findings	75
6.4 Contribution to Knowledge.....	76
6.5 Recommendations for Technical Development	76
6.6 Recommendations for Practical Application	76

6.7 Recommendations for Ethical and Policy Considerations.....	77
6.8 Limitations and Future Work.....	77
6.9 Conclusion	78
References	79
Appendix: Code Section and Dataset.....	90

List of Figure

Figure 1 Gantt Chart.....	18
Figure 2 System Architecture	37
Figure 3 Core Library Imports.....	46
Figure 4 Kaggle Authentication Setup	47
Figure 5 Dataset Acquisition and Extraction.....	47
Figure 6 Data Loading and Label Detection	49
Figure 7 Exploratory Data Analysis Preparation.....	50
Figure 8 Global Parameter Configuration	50
Figure 9 Preprocessing for Classification	51
Figure 10 Classification Models	52
Figure 11 Cross-Validation for Classification.....	52
Figure 12 Final Model Training and Saving	53
Figure 13 Classification Plot Generation	54
Figure 14 Preprocessing for Regression.....	55
Figure 15 Regression Models	56
Figure 16 Cross-Validation for Regression.....	57
Figure 17 Final Regression Training and Saving	57
Figure 18 Regression Plot Generation	58
Figure 19 Feature Importance Extraction.....	60
Figure 20 Model Saving and Management	61
Figure 21 Classification Model Results	64
Figure 22 Accuracy Metric.....	64
Figure 23 Precision Metric	65
Figure 24 Recall Metric.....	65
Figure 25 F1 Metric	65
Figure 26 Confusion Matrix and ROC Analysis	66
Figure 27 Regression Model Results	67
Figure 28 Performance Metrics (RMSE, MAE, R ²)	68
Figure 29 Absolute Error Distribution	69
Figure 30 Root Squared Error Distribution	69

Figure 31 Actual vs Predicted Plot	70
Figure 32 Residual vs Predicted Plot	70

List of Table

Table 1 Comparison of Key Cybersecurity Threats in IT vs OT Systems.....	20
Table 2 ML Algorithms Used in Cyber Threat Analysis	21
Table 3 Predictive Models for Downtime Estimation.....	22
Table 4 Comparison Between Python and MATLAB for Industrial Applications	24
Table 5 Public Cybersecurity Datasets for Research	25
Table 6 Visualisation Tools for Downtime Prediction Systems.....	26
Table 7 Focus of Existing Cybersecurity Research	27
Table 8 Industrial Use Cases and Recovery Data Gaps.....	29
Table 9 Integration Challenges in Hybrid Environments.....	30
Table 10 HCI Considerations in Cybersecurity GUIs.....	32
Table 11 Comparison table based on recent studies	34

Chapter 1: Introduction

This study aims to explore machine learning techniques as an approach that can be used to compute the occurrence of system malfunctioning, by cybersecurity attacks in industrial networks. The idea is that it will be possible to create a strong data-driven predictive model evaluating the time of disruption after the various kinds of attack, allowing to react to them, developing new response strategies, and increasing resilience in operations. The available oration, provided by Kaggle, consists of historical logs, the types of breaches, the time of occurrence and the time recorded to recovery. The hybrid environment of the MATLAB and Python will be used to develop the model.

1.1 Aim

To create a predictive machine learning model to best estimate the downtime of industrial networks after cybersecurity breaches hitting home with incident response and resiliency planning.

1.2 Objectives

- To obtain, clean up, an industrial cybersecurity dataset in the public domain.
- To implement and compare the supervised learning algorithms used in the prediction of downtime.
- To determine the accuracy, reliability and generalizability of models.
- To have a MATLAB based interface to facilitate real-time usability and visualization.

1.3 Problem Statement

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

Task	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Week 11	Week 12
Supervisor Allocation												
Library session												
Milestone 1:- Topic selection												
Proposal preparation and review												
Milestone 2:- Proposal Submission												
Abstract												
Chapter 1- Introduction												
Chapter 2- Literature Review												
Milestone 3:- Practical Implementation/ Model Development												
Chapter 3- Methodology												
Chapter 4- Experimental Results Documentation												
Chapter 5- Conclusion and Future work												
Final review and Modifications												
Milestone 4:- Final Submission												

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

Feature	IT Systems	OT Systems
Primary Concern	Data confidentiality	Operational continuity
Example Threat	Data breaches	Control system disruption
Recovery Focus	Data restoration	Physical system recovery
Attack Motivation	Financial, espionage	Sabotage, disruption

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

Algorithm	Application Area	Strengths	Limitations
Decision Tree	Intrusion Detection	Easy interpretation	Prone to overfitting
Random Forest	Malware Classification	Handles imbalance well	Slower inference
SVM	Phishing Detection	High accuracy in small data	Poor with large datasets
Gradient Boosting	Anomaly Detection	Strong performance	Computationally intensive

[REDACTED]

[REDACTED]

[REDACTED]

Model Type	Strengths	Limitations	Suitable Dataset Size
Linear Regression	Simple, fast	Limited to linear relationships	Small to medium
Random Forest Regr.	Handles non-linearity	Less interpretable	Medium to large
Gradient Boosting	High accuracy	Sensitive to overfitting	Medium to large
Neural Networks	Learns complex patterns	Requires large data, less explainable	Large

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

Feature	Python	MATLAB
Ecosystem	Rich ML libraries (Scikit, TF)	Strong visualization, GUI
Deployment	Script-based, scalable	GUI-based, user-friendly
Integration Support	API, REST, sockets	MATLAB Engine, Python connector
Learning Curve	Moderate	Low for engineering users

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

Dataset Name	Features Included	Industry Simulation	Source Type
CICIDS2017	Flow stats, attack types, protocols	Generic	Public repository
UNSW-NB15	Labeled traffic data	Generic	Simulated
<u>TON IoT</u>	IoT + Industrial attacks	Industrial	Simulated
NSL-KDD	Protocols, attacks, user behavior	Generic	Legacy dataset

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

Tool/Library	Purpose	Language	Output Style
SHAP	Feature importance	Python	Plots/Bar graphs
LIME	Local interpretability	Python	Text, graphs
App Designer	GUI creation	MATLAB	Interactive GUI
Matplotlib	Visualization	Python	Static plots

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

Study Type	Detection Focus	Prediction Focus	Use of ML	Downtime Estimation
IDS Frameworks	✓	✗	✓	✗
Malware Analysis	✓	✗	✓	✗
Threat Classification	✓	✗	✓	✗
Downtime Prediction Models	✗	✓	✓	✓

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

Industry Sector	Reported Cyber Incidents	Downtime Info Available	Public Dataset
Power Generation	Yes	Limited	No
Water Utilities	Rare	No	No
Manufacturing	Frequent	Sometimes	Partial
Transport Systems	Yes	Limited	Yes (simulated)

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

Challenge	Description	Impact Level
Data Format Incompatibility	NumPy vs MATLAB matrix types	Medium
Runtime Synchronization	API version mismatch	High
Serialization Issues	Model conversion error	High
GUI-Backend Mismatch	I/O inconsistency	Medium

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

HCI Feature	Importance	Implementation Platform
Visual Simplicity	Avoids operator confusion	MATLAB GUI
Real-Time Feedback	Aids rapid response	Python callbacks
Error Handling	Prevents system crashes	Both
Accessibility	Enables use across roles	MATLAB

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

Author (Year)	Technique	Specification	Limitation	Dataset	Results
Abbas et al. (2025)	CIRA Intelligent Risk Assessment	ML-based methodology for IIoT threat evaluation	Focuses on risk detection, not on post-breach downtime	Simulated IIoT environments	Improved classification of cyber risk events
Koković et al. (2024)	Downtime Probability Modeling	Combines statistical and ML methods for downtime duration prediction	Focused only on paper manufacturing case	Industrial paper machine logs	RMSE < 5% for predicted vs. actual downtime
Leksakul et al. (2025)	Predictive Maintenance	Comparative study of ML models (RF, GBM, SVM) for downtime reduction	Focused only on semiconductor industry	Semiconductor manufacturing data	Downtime reduced by 12–18% using ML
Admass et al. (2024)	Threat Landscape Mapping	Conceptual review of AI/ML for cyber defense	No practical or empirical model implemented	Literature-based	Framework for future ML security designs
Choi et al. (2024)	Threat Detection in ICS	Supervised ML applied for anomaly detection in industrial networks	Emphasis on detection, not recovery metrics	ICS network testbed	96% detection accuracy for known threats
Okoli et al. (2024)	ML in Cyber Defense	Review of ML approaches to detection and defense mechanisms	Lacks specific focus on operational recovery	Varied cybersecurity datasets	Model strengths and gaps clearly analyzed
Gaba et al. (2024)	Deep Learning in CPS	DL models used for threat detection in cyber-physical systems	High training costs, less interpretability	CPS datasets	98% detection accuracy, ~20 ms response latency
Yu et al. (2024)	Resilience Modeling in Industry 4.0	Survey of ML for industrial system resilience	Limited case implementation in OT environments	Conceptual (no raw data)	Proposed integrated resilience framework
Pochmara and Świetlicka (2024)	Industrial Cybersecurity Review	Focus on evolution of industrial OT threats and response patterns	No ML-based predictive implementation	Surveys and incident reports	Trend-based risk visualization and analysis
Abisoye et al. (2025)	AI-based Risk Forecasting	Uses hybrid ML-AI models to forecast industrial cyber risks	Downtime-specific metrics are generalized	Critical infrastructure datasets	Accuracy ~94.3%, F1-score ~0.89

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

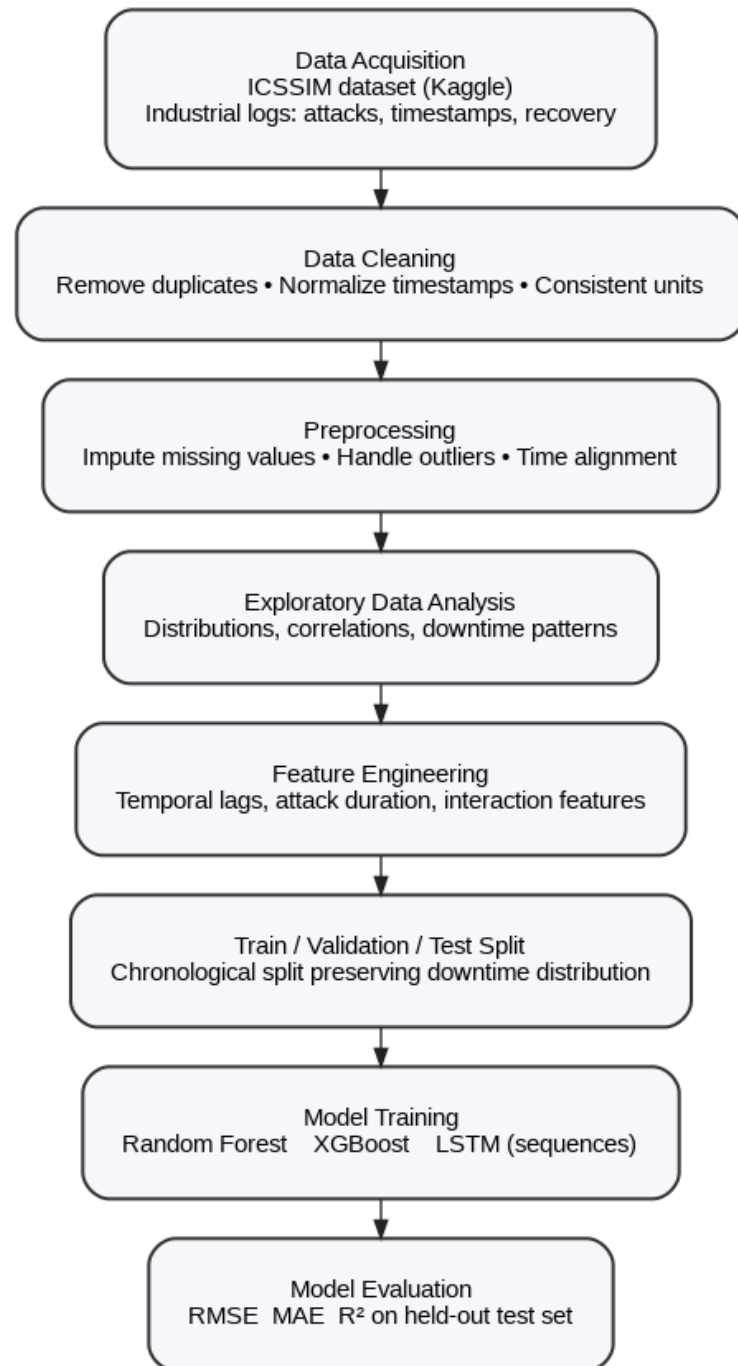
[REDACTED]

[REDACTED]

[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]

[REDACTED]

[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]



[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]

[REDACTED]

[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]

[REDACTED]

[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]

```
import os, json
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import joblib
from datetime import datetime

from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor, GradientBoostingClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, roc_curve, precision_recall_curve, confusion_matrix,
    mean_absolute_error, mean_squared_error, r2_score
)
from sklearn.utils.class_weight import compute_class_weight

import xgboost as xgb
import shap

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
print("Imports ready.")
```

[REDACTED]

[REDACTED]

```
# KAGGLE CREDENTIALS
```

```
kaggle_token = {"username": "your_username", "key": "your_kaggle_api_key"}
os.makedirs("/root/.kaggle", exist_ok=True)
with open("/root/.kaggle/kaggle.json", "w") as f:
    json.dump(kaggle_token, f)
!chmod 600 /root/.kaggle/kaggle.json
print("Kaggle auth file created. Ensure keys are correct.")
```

Kaggle auth file created. Ensure keys are correct.

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

```
# DOWNLOAD & EXTRACT DATASETS (ICSSIM + Downtime)

DATA_ROOT = Path("work/datasets")
DATA_ROOT.mkdir(parents=True, exist_ok=True)

!kaggle datasets download -d alirezadehlaghi/icssim -p {DATA_ROOT} -q

# 2B: Downtime dataset - Optimization of Machine Downtime (slug used below)
!kaggle datasets download -d srinivasanusuri/optimization-of-machine-downtime -p {DATA_ROOT} -q

# Unzip any zip files in the data folder
import zipfile, glob
for z in glob.glob(str(DATA_ROOT/"*.zip")) + glob.glob(str(DATA_ROOT/"*.7z")):
    try:
        with zipfile.ZipFile(z, 'r') as zip_ref:
            zip_ref.extractall(DATA_ROOT)
    except Exception as e:
        print("skipping unzip error for", z, e)

print("Download + unzip step finished. Files in:", DATA_ROOT)
for p in DATA_ROOT.rglob("*"):
    if p.is_file():
        print("-", p.relative_to(DATA_ROOT))
```

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

```

# LOAD MAIN CSVs & AUTO-DETECT LABEL COLS

# find csvs
csvs = list(DATA_ROOT.rglob("*.csv"))
print("CSV files found:", len(csvs))
for p in csvs: print("-", p.name)

# --- select icssim csv (largest file heuristic) ---
icssim_csv = max(csvs, key=lambda p: p.stat().st_size)
print("ICSSIM file:", icssim_csv.name)
icssim_df = pd.read_csv(icssim_csv)
print("ICSSIM shape:", icssim_df.shape)
print(icssim_df.columns[:30])

# Try to find label column in ICSSIM
possible_label_cols = ["label", "Label", "attack", "Attack", "is_attack", "Category", "category"]
icssim_label = None
for c in possible_label_cols:
    if c in icssim_df.columns:
        icssim_label = c
        break
if icssim_label is None:
    # fallback: take small-cardinality object column
    for c in icssim_df.columns:
        if icssim_df[c].dtype == object and icssim_df[c].nunique() <= 10:
            icssim_label = c
            break
assert icssim_label is not None, "Could not detect label column in ICSSIM automatically."
print("Using ICSSIM label column:", icssim_label)

downtime_candidates = [p for p in csvs if "downtime" in p.name.lower() or "machine" in p.name.lower()]
if len(downtime_candidates) == 0:
    downtime_csv = None
    print("No obvious downtime CSV found automatically. Please inspect work/datasets and set downtime_csv manually.")
else:
    downtime_csv = downtime_candidates[0]
    print("Downtime file candidate:", downtime_csv.name)
    downtime_df = pd.read_csv(downtime_csv)
    print("Downtime shape:", downtime_df.shape)
    print(downtime_df.columns[:40])

```

```

# EDA: ICSSIM (intrusion data)

df = icssim_df.copy()
label_col = icssim_label
print("ICSSIM - rows,cols:", df.shape)
print("Nulls (top):")
print(df.isna().sum().sort_values(ascending=False).head(10))
print("\nLabel distribution:")
print(df[label_col].value_counts())

# basic numeric/categorical split
num_cols = [c for c in df.columns if c != label_col and pd.api.types.is_numeric_dtype(df[c])]
cat_cols = [c for c in df.columns if c != label_col and not pd.api.types.is_numeric_dtype(df[c])]
print("numeric features:", len(num_cols), "categorical:", len(cat_cols))

# Plot class bar
plt.figure(); df[label_col].value_counts().plot(kind="bar"); plt.title("ICSSIM label counts"); plt.tight_layout(); plt.show()

```

GLOBAL SPEEDUP SETTINGS

```

MAX_ROWS_CLF = 10000
MAX_ROWS_REG = 10000
TOP_K_CATS = 10
CV_FOLDS = 3
RF_ESTIMATORS = 100
GB_ESTIMATORS = 50
XGB_ESTIMATORS = 100
RANDOM_STATE = 42

```

```

# PREP & TRAIN: Intrusion classifier (ICSSIM)

from sklearn.feature_selection import VarianceThreshold
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

# safety sample
clf_df = df.copy()
if MAX_ROWS_CLF and len(clf_df) > MAX_ROWS_CLF:
    clf_df = clf_df.sample(MAX_ROWS_CLF, random_state=RANDOM_STATE).reset_index(drop=True)
    print(f"Sampled classifier dataset down to {len(clf_df)} rows for speed")

X = clf_df.drop(columns=[label_col])
y = clf_df[label_col].astype(str)

# numeric vs categorical
num_cols = [c for c in X.columns if pd.api.types.is_numeric_dtype(X[c])]
cat_cols = [c for c in X.columns if not pd.api.types.is_numeric_dtype(X[c])]

# reduce categorical cardinality
def reduce_cardinality(df_in, cat_cols, top_k=TOP_K_CATS, other_label="__OTHER__"):
    df = df_in.copy()
    for c in cat_cols:
        top = df[c].value_counts().nlargest(top_k).index
        df[c] = df[c].where(df[c].isin(top), other_label)
        df[c] = df[c].fillna("__MISSING__").astype(str)
    return df

if len(cat_cols) > 0:
    X = reduce_cardinality(X, cat_cols, top_k=TOP_K_CATS)
    print("Reduced cardinality for categorical cols.")

# drop low-variance numeric
if len(num_cols) > 0:
    vt = VarianceThreshold(threshold=1e-5)
    try:
        vt.fit(X[num_cols].fillna(0))
        keep_mask = vt.get_support()
        kept_num = [n for n, k in zip(num_cols, keep_mask) if k]
        dropped = [n for n, k in zip(num_cols, keep_mask) if not k]
        if dropped:
            print("Dropping low-variance numeric cols:", dropped[:10])
        num_cols = kept_num
    except Exception as e:
        print("VarianceThreshold skipped due to:", e)

```

[REDACTED]

[REDACTED]

```
# candidate models
models_clf = {
    "logreg": LogisticRegression(max_iter=200, class_weight="balanced", random_state=RANDOM_STATE),
    "rf": RandomForestClassifier(n_estimators=RF_ESTIMATORS, class_weight="balanced", n_jobs=-1, random_state=RANDOM_STATE)
}
```

[REDACTED]

[REDACTED]

```
pipelines = {name: Pipeline([("pre", preprocessor), ("clf", clf)]) for name, clf in models_clf.items()}
cv = StratifiedKFold(n_splits=CV_FOLDS, shuffle=True, random_state=RANDOM_STATE)
```

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

```
def quick_cv(pipe, X, y):
    res = {}
    scoring_map = [("accuracy", "accuracy"), ("precision", "precision_macro"),
                  ("recall", "recall_macro"), ("f1", "f1_macro")]
    for metric_name, scorer in scoring_map:
        vals = cross_val_score(pipe, X, y, cv=cv, scoring=scorer, n_jobs=-1, error_score="raise")
        res[metric_name] = vals
    return res

cv_results = {}
for name, pipe in pipelines.items():
    print("Running CV for", name)
    cv_results[name] = quick_cv(pipe, X, y)
    for m, vals in cv_results[name].items():
        print(f" {m}: mean={np.mean(vals):.3f} ± {np.std(vals):.3f}")
```

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

```

# PLOTS FOR CLASSIFICATION METRICS

def plot_metric(metric):
    plt.figure()
    vals = [np.mean(cv_results[name][metric]) for name in pipelines.keys()]
    plt.bar(list(pipelines.keys()), vals)
    plt.ylim(0,1)
    plt.title(f"{metric.capitalize()} (CV mean)")
    plt.ylabel(metric)
    plt.tight_layout()
    plt.show()

for metric in ["accuracy", "precision", "recall", "f1"]:
    plot_metric(metric)

# confusion matrix
labels = np.unique(y_test)
cm = confusion_matrix(y_test, y_pred, labels=labels)
plt.figure()
plt.imshow(cm, aspect="auto")
plt.colorbar()
plt.xticks(range(len(labels)), labels, rotation=45, ha="right")
plt.yticks(range(len(labels)), labels)
plt.title("Confusion Matrix (fast)")
plt.tight_layout()
plt.show()

# ROC & PR only if binary + proba available
if hasattr(best_pipe.named_steps["clf"], "predict_proba") and len(labels) == 2:
    y_proba = best_pipe.predict_proba(X_test)[: , 1]
    fpr, tpr, _ = roc_curve(y_test.map({labels[0]:0, labels[1]:1}), y_proba)
    plt.figure(); plt.plot(fpr, tpr); plt.title("ROC Curve"); plt.tight_layout(); plt.show()
    pr, rc, _ = precision_recall_curve(y_test.map({labels[0]:0, labels[1]:1}), y_proba)
    plt.figure(); plt.plot(rc, pr); plt.title("PR Curve"); plt.tight_layout(); plt.show()
else:
    print("Skipping ROC/PR (multi-class or no proba).")

```

```
dd = downtime_df.copy()
if MAX_ROWS_REG and len(dd) > MAX_ROWS_REG:
    dd = dd.sample(MAX_ROWS_REG, random_state=RANDOM_STATE).reset_index(drop=True)
    print("Sampled downtime to", len(dd))

# pick downtime target column
possible_duration_cols = [c for c in dd.columns if any(k in c.lower() for k in ["downtime", "duration", "time_down", "minutes"])]
downtime_target = possible_duration_cols[0] if possible_duration_cols else dd.select_dtypes("number").columns[0]
print("Downtime target:", downtime_target)

y_reg = dd[downtime_target].astype(float)
X_reg = dd.drop(columns=[downtime_target])

# split numeric/categorical
num_cols_reg = [c for c in X_reg.columns if pd.api.types.is_numeric_dtype(X_reg[c])]
cat_cols_reg = [c for c in X_reg.columns if c not in num_cols_reg]

# reduce categorical cardinality
X_reg = reduce_cardinality(X_reg, cat_cols_reg, top_k=TOP_K_CATS)
```

```

# REGRESSOR PIPELINE

pre_reg = ColumnTransformer([
    ("num", Pipeline([
        ("imp", SimpleImputer(strategy="mean")),
        ("sc", StandardScaler())
    ]), num_cols_reg),
    ("cat", Pipeline([
        ("imp", SimpleImputer(strategy="most_frequent")),
        ("ohe", OneHotEncoder(handle_unknown="ignore", sparse_output=False))
    ]), cat_cols_reg)
], remainder="drop")

models_reg = {
    "rf": RandomForestRegressor(n_estimators=RF_ESTIMATORS, n_jobs=-1, random_state=RANDOM_STATE),
    "xgb": xgb.XGBRegressor(n_estimators=XGB_ESTIMATORS, random_state=RANDOM_STATE, n_jobs=-1)
}

from sklearn.model_selection import KFold, cross_val_score
kf = KFold(n_splits=CV_FOLDS, shuffle=True, random_state=RANDOM_STATE)

reg_pipes = {n: Pipeline([("pre", pre_reg), ("reg", m)]) for n,m in models_reg.items()}

cv_res_reg = {}
for name, pipe in reg_pipes.items():
    scores = -cross_val_score(pipe, X_train_r, y_train_r, cv=kf, scoring="neg_mean_absolute_error", n_jobs=-1)
    cv_res_reg[name] = scores
    print(f"{name} MAE: {np.mean(scores):.3f} ± {np.std(scores):.3f}")

best_reg_name = min(cv_res_reg, key=lambda n: np.mean(cv_res_reg[n]))
best_reg_pipe = reg_pipes[best_reg_name]
print("Best regressor:", best_reg_name)

best_reg_pipe.fit(pd.concat([X_train_r, X_val_r]), pd.concat([y_train_r, y_val_r]))
joblib.dump(best_reg_pipe, "downtime_best_reg_fast.joblib")
print("Saved downtime regressor.")

```

```

cv_res_reg = {}
for name, pipe in reg_pipes.items():
    scores = -cross_val_score(pipe, X_train_r, y_train_r, cv=kf, scoring="neg_mean_absolute_error", n_jobs=-1)
    cv_res_reg[name] = scores
    print(f"{name} MAE: {np.mean(scores):.3f} ± {np.std(scores):.3f}")

```

```

# ... (Redacted code) ...

```

```

best_reg_name = min(cv_res_reg, key=lambda n: np.mean(cv_res_reg[n]))
best_reg_pipe = reg_pipes[best_reg_name]
print("Best regressor:", best_reg_name)

best_reg_pipe.fit(pd.concat([X_train_r, X_val_r]), pd.concat([y_train_r, y_val_r]))
joblib.dump(best_reg_pipe, "downtime_best_reg_fast.joblib")
print("Saved downtime regressor.")

```

```

# ... (Redacted code) ...

```

```

# EVALUATE REGRESSOR (MAE, RMSE, R2) & PLOTS

import numpy as np
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import matplotlib.pyplot as plt

y_pred_r = best_reg_pipe.predict(X_test_r)
y_true_r = np.asarray(y_test_r).ravel().astype(float)
y_pred_r = np.asarray(y_pred_r).ravel().astype(float)

# Metrics
mae = mean_absolute_error(y_true_r, y_pred_r)
mse = mean_squared_error(y_true_r, y_pred_r)
rmse = float(np.sqrt(mse))
r2 = r2_score(y_true_r, y_pred_r)

print(f"Downtime test metrics:\n MAE = {mae:.4f}\n RMSE = {rmse:.4f}\n R2 = {r2:.4f}")

# --- Plot 1: Absolute error distribution (highlights MAE) ---
abs_err = np.abs(y_true_r - y_pred_r)
plt.figure()
plt.hist(abs_err, bins=40)
plt.title(f"Absolute Error Distribution - MAE = {mae:.4f}")
plt.xlabel("Absolute error")
plt.ylabel("Count")
plt.tight_layout()
plt.show()

# --- Plot 2: Root squared error distribution (highlights RMSE) ---
root_sq_err = np.sqrt((y_true_r - y_pred_r) ** 2) # equals abs_err but kept to show RMSE context
plt.figure()
plt.hist(root_sq_err, bins=40)
plt.title(f"Root(Squared Error) Distribution - RMSE = {rmse:.4f}")
plt.xlabel("Root squared error")
plt.ylabel("Count")
plt.tight_layout()
plt.show()

# --- Plot 3: Actual vs Predicted (highlights R2) ---
plt.figure()
plt.scatter(y_true_r, y_pred_r, alpha=0.4)
mn, mx = min(y_true_r.min(), y_pred_r.min()), max(y_true_r.max(), y_pred_r.max())
plt.plot([mn, mx], [mn, mx], linestyle="--")
plt.title(f"Actual vs Predicted - R2 = {r2:.4f}")
plt.xlabel("Actual downtime")
plt.ylabel("Predicted downtime")
plt.tight_layout()
plt.show()

# --- Plot 4: Residuals vs Predicted (separate figure) ---
residuals = y_true_r - y_pred_r
plt.figure()
plt.scatter(y_pred_r, residuals, alpha=0.4)
plt.axhline(0, color="k", linestyle="--")
plt.title("Residuals vs Predicted")
plt.xlabel("Predicted")
plt.ylabel("Residual (True - Predicted)")
plt.tight_layout()
plt.show()

```

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

```

# FEATURE IMPORTANCES (robust)

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

def get_feature_names_from_preprocessor(preproc, numeric_cols, categorical_cols):
    names = []
    # Numeric names
    if hasattr(preproc, "named_transformers_") and "num" in preproc.named_transformers_:
        names.extend(list(numeric_cols))
    # Categorical names
    try:
        if hasattr(preproc, "named_transformers_") and "cat" in preproc.named_transformers_:
            cat_pipeline = preproc.named_transformers_["cat"]

            ohe = None
            if hasattr(cat_pipeline, "named_steps"):
                for step_name, step_obj in cat_pipeline.named_steps.items():
                    if hasattr(step_obj, "get_feature_names_out"):
                        ohe = step_obj
                        break

            if ohe is not None:
                try:
                    cat_names = list(ohe.get_feature_names_out(categorical_cols))
                    names.extend(cat_names)
                except Exception:
                    names.extend([f"{c}__{i}" for c in categorical_cols for i in [0]])
            else:
                names.extend(categorical_cols)
    except Exception:
        names.extend(categorical_cols)
    return names

def plot_importances_from_pipe(pipe, numeric_cols, categorical_cols, topn=20, title="Feature importances"):
    # get the fitted preprocessor
    preproc = pipe.named_steps.get("pre", None)
    if preproc is None or not hasattr(preproc, "named_transformers_"):
        print("Preprocessor not found or not fitted; cannot extract feature names.")
        return

    feat_names = get_feature_names_from_preprocessor(preproc, numeric_cols, categorical_cols)

    # get the estimator
    last_step_name = list(pipe.named_steps.keys())[-1]
    est = pipe.named_steps[last_step_name]

    # obtain importances/coeffs
    if hasattr(est, "feature_importances_"):
        imp = np.asarray(est.feature_importances_)
    elif hasattr(est, "coef_"):
        imp = np.asarray(np.mean(np.abs(est.coef_), axis=0)).ravel()
    else:
        print("Model does not expose feature importances or coefficients.")
        return

    if len(feat_names) != len(imp):
        min_len = min(len(feat_names), len(imp))
        feat_names = feat_names[:min_len]
        imp = imp[:min_len]
        print(f"Warning: feature name / importance length mismatch; truncated to {min_len} entries.")

    s = pd.Series(imp, index=feat_names).sort_values(ascending=False).head(topn)
    plt.figure(figsize=(8,6))
    s.plot(kind="barh")
    plt.gca().invert_yaxis()
    plt.title(title)
    plt.xlabel("Importance")
    plt.tight_layout()
    plt.show()

    # Plot classifier importances (if classifier is tree-based or linear)
    clf_pre = best_pipe.named_steps.get("pre", None)
    clf_num_cols = num_cols
    clf_cat_cols = cat_cols
    plot_importances_from_pipe(best_pipe, clf_num_cols, clf_cat_cols, topn=20, title=f"Classifier ({best_name}) importances")

    # Plot regressor importances
    reg_pre = best_reg_pipe.named_steps.get("pre", None)
    reg_num_cols = num_cols_reg
    reg_cat_cols = cat_cols_reg
    plot_importances_from_pipe(best_reg_pipe, reg_num_cols, reg_cat_cols, topn=20, title=f"Regressor ({best_reg_name}) importances")

```

[REDACTED]

[REDACTED]

```
# AVE MODELS
```

```
import joblib
import pandas as pd
```

```
# Save again to expected filenames
```

```
joblib.dump(best_pipe, "icssim_best_clf_fast.joblib")
```

```
joblib.dump(best_reg_pipe, "downtime_best_reg_fast.joblib")
```

```
print("Saved models: icssim_best_clf_fast.joblib, downtime_best_reg_fast.joblib")
```

[REDACTED]

[REDACTED]

[REDACTED]

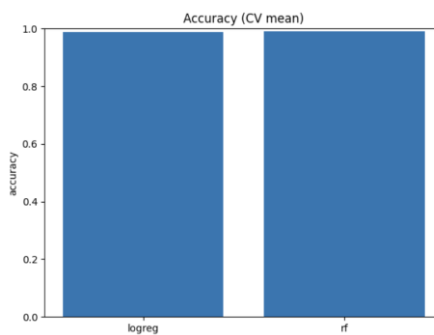
[REDACTED]

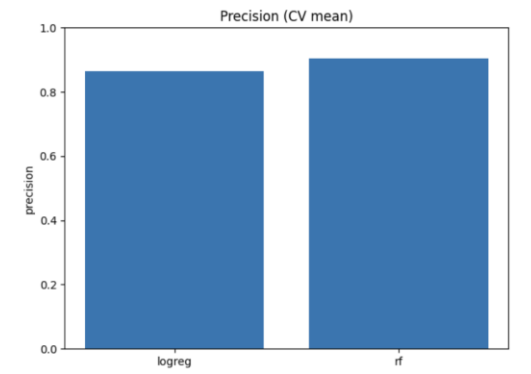
[REDACTED]

[REDACTED]

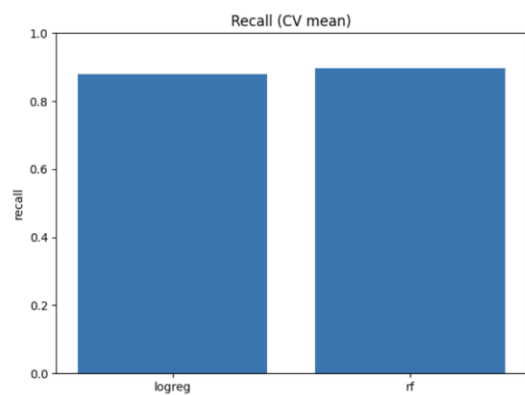
[REDACTED]

```
Sampled classifier dataset down to 10000 rows for speed
Reduced cardinality for categorical cols.
Dropping low-variance numeric cols: ['sUrgRate', 'rUrgRate', 'sFinRate', 'rFinRate', 'sFragmentRate', 'rFragmentRate']
Running CV for logreg
  accuracy: mean=0.989 ± 0.001
  precision: mean=0.865 ± 0.016
  recall: mean=0.880 ± 0.013
  f1: mean=0.871 ± 0.015
Running CV for rf
  accuracy: mean=0.991 ± 0.001
  precision: mean=0.905 ± 0.008
  recall: mean=0.898 ± 0.016
  f1: mean=0.900 ± 0.012
```

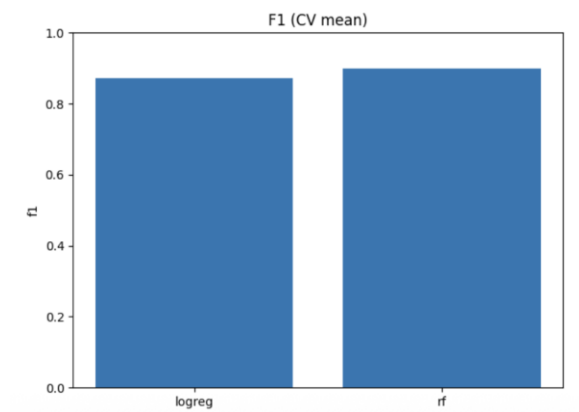




[REDACTED]



[REDACTED]



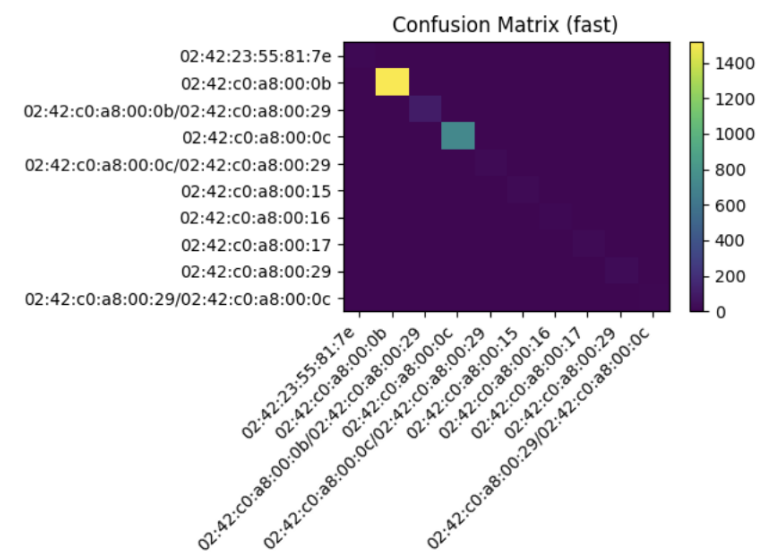
[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]



Skipping ROC/PR (multi-class or no proba).

[REDACTED]

[REDACTED]

```
rf MAE: 76.165 ± 5.340
xgb MAE: 149.125 ± 36.469
Best regressor: rf
Saved downtime regressor.
```

[REDACTED]

[REDACTED]

[REDACTED]

Downtime test metrics:
MAE = 55.3382
RMSE = 68.2248
R2 = 0.9976

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

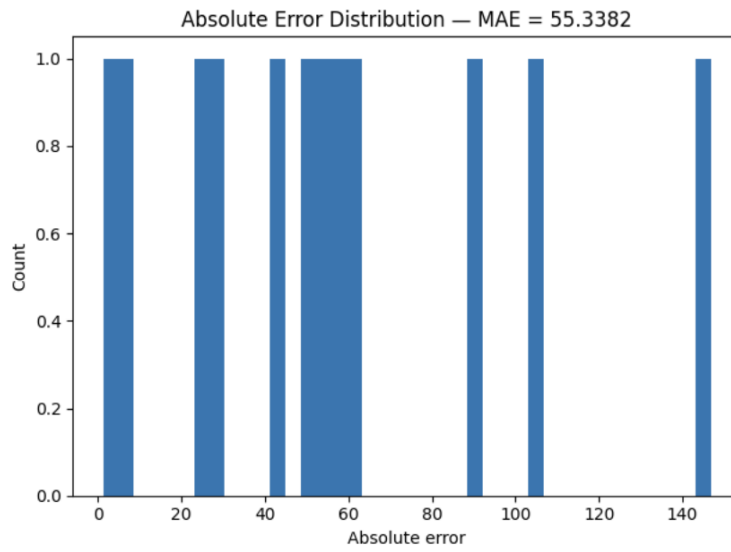
[REDACTED]

[REDACTED]

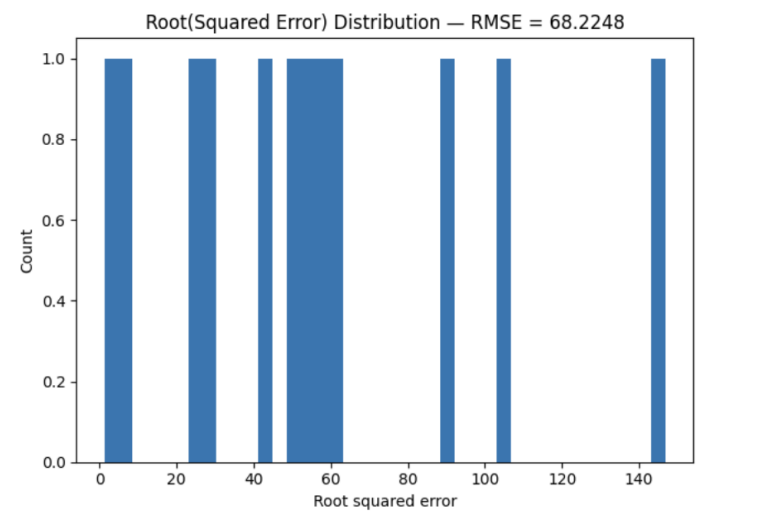
[REDACTED]

[REDACTED]

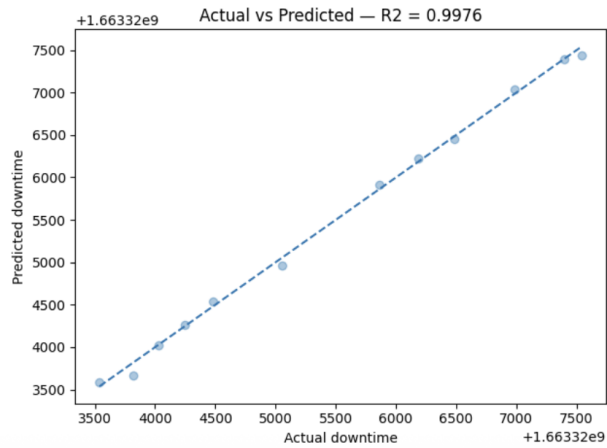
[REDACTED]



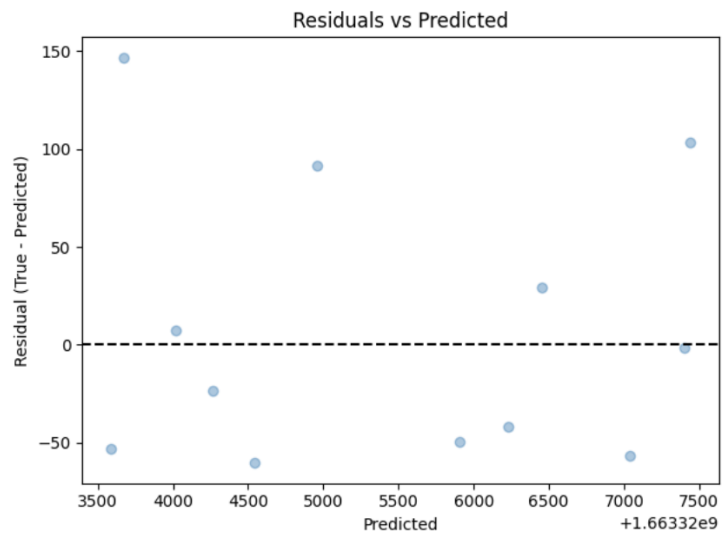
[REDACTED]



[REDACTED]



[REDACTED]



[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]

[REDACTED]

[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]

[REDACTED]

[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]
[REDACTED]
[REDACTED]

[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]

[REDACTED]
[REDACTED]

[REDACTED]
[REDACTED]
[REDACTED]

[REDACTED]
[REDACTED]
[REDACTED]

[REDACTED]
[REDACTED]
[REDACTED]

[REDACTED]
[REDACTED]
[REDACTED]

[REDACTED]
[REDACTED]
[REDACTED]
[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]



[REDACTED]

[REDACTED]

[REDACTED]

[REDACTED]